

Guide d'Onboarding - Nymphéa

Documentation pour les développeurs rejoignant le projet de migration e-commerce Salesforce → CommerceTools + Contentstack

Table des matières

- 1. [Vue d'ensemble du projet](#)
- 2. [Stack technique et choix d'architecture](#)
- 3. [Structure du projet](#)
- 4. [Astro : le framework](#)
- 5. [Preact Signals : état client](#)
- 6. [Récupération des données](#)
- 7. [Connexion CommerceTools](#)
- 8. [Live Preview Contentstack](#)
- 9. [Méthodologie d'implémentation](#)

1. Vue d'ensemble du projet

Contexte

POC de migration de nos sites e-commerce (Etam, Undiz, Maison123) de **Salesforce** vers :

- **CommerceTools** : gestion produits, panier, commandes
- **Contentstack** : gestion du contenu éditorial (CMS headless)

Objectifs techniques

- **Performance** : SSR avec hydratation partielle (Islands Architecture)
- **Multi-marques** : un seul codebase, thèmes par marque
- **Édition live** : les contributeurs voient leurs modifications en temps réel
- **Migration progressive** : passage vers BFF (Back-to-Front) à terme

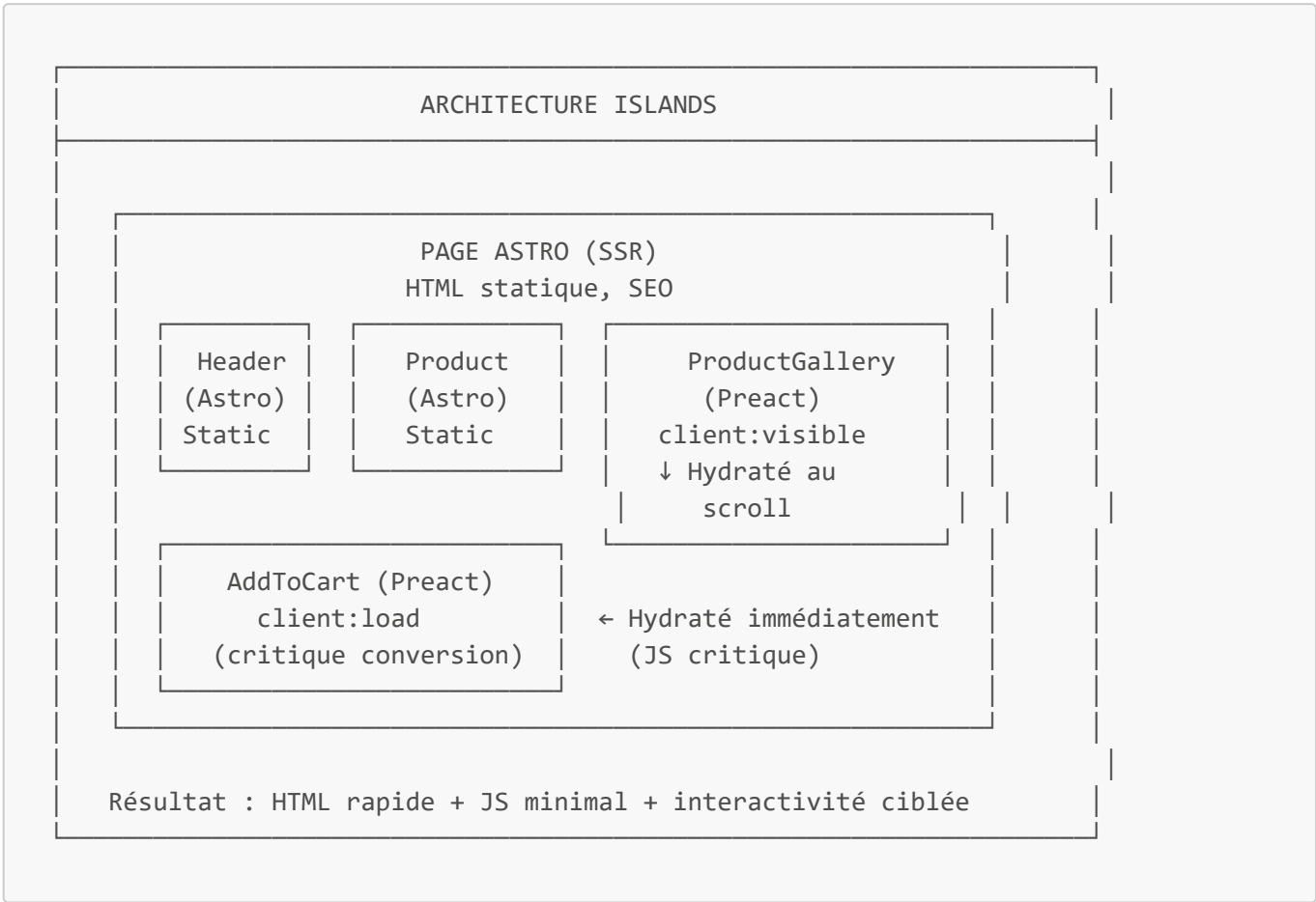
2. Stack technique et choix d'architecture

Stack

Technologie	Rôle	Pourquoi ?
Astro	Framework SSR	Islands Architecture, excellent SEO, bundle minimal
Preact	UI Components	Compatible React, 3KB, performant
Preact Signals	State Management	Réactif, simple, pas de boilerplate
Tailwind CSS v4	Styling	Design tokens, theming multi-marques
CommerceTools	Backend e-commerce	API-first, scalable

Technologie	Rôle	Pourquoi ?
Contentstack	CMS Headless	Live Preview, Visual Builder

Pourquoi Astro + Preact ?



Avantages :

- 🚀 **First Paint ultra-rapide** : HTML pré-rendu côté serveur
- 📦 **Bundle JS minimal** : seuls les composants interactifs sont hydratés
- 🔗 **SEO natif** : contenu indexable sans JavaScript
- ⚡ **Hydratation sélective** : `client:load`, `client:visible`, `client:idle`

3. Structure du projet

```
src/
├── assets/                                # Icons par marque
│   └── icons/{brand}/
├── components/                           # Design System (60+ composants)
│   ├── Button.tsx
│   ├── Button.props.ts
│   └── ...
└── config/
```

└─ brands.ts	# Configuration multi-marques
└─ features/	# Features métier
│ └─ auth/	# Authentification
│ └─ cart/	# Panier
│ └─ product/	# Fiche produit
│ └─ category/	# Listing produits
│ └─ homepage/	# Page d'accueil
│ └─ layout/	# Header, Footer, Navigation
└─ layouts/	
│ └─ Layout.astro	# Layout principal
└─ lib/	
│ └─ api/	# Appels HTTP bruts
│ │ └─ commercetools.ts	# ⚠ SERVER-ONLY
│ │ └─ contentstack.ts	# ⚠ SERVER-ONLY
│ └─ domain/	# Transformation des données
│ │ └─ product.ts	# CT → UI
│ │ └─ cart.ts	
│ │ └─ content.ts	# Contentstack → UI
│ └─ services/	# Orchestration API + Domain
│ │ └─ product.service.ts	
│ │ └─ cart.service.ts	
│ │ └─ content.service.ts	
│ └─ signals/	# State client (Preact Signals)
│ │ └─ cart.store.ts	
│ │ └─ auth.store.ts	
│ │ └─ wishlist.store.ts	
│ └─ contentstack-sdk/	# Live Preview hooks
│ │ └─ useLivePreview.ts	
└─ i18n/	# Internationalisation
└─ pages/	
│ └─ index.astro	# Homepage
│ └─ product/[id].astro	# Fiche produit (route dynamique)
│ └─ cart.astro	# Panier
│ └─ api/	# API Routes (endpoints internes)
│ │ └─ cart/	
│ │ └─ auth/	
│ │ └─ products/	
└─ styles/	
│ └─ themes/	# Tokens par marque
│ │ └─ etam.css	
│ │ └─ undiz.css	
│ │ └─ maison123.css	
│ └─ components/	# CSS par composant

```
└─ utils/  
  └─ base-path.ts          # Gestion du base path
```

4. Astro : le framework

Principe de base

Astro génère du **HTML statique par défaut**. L'interactivité est ajoutée uniquement là où c'est nécessaire via les **Islands**.

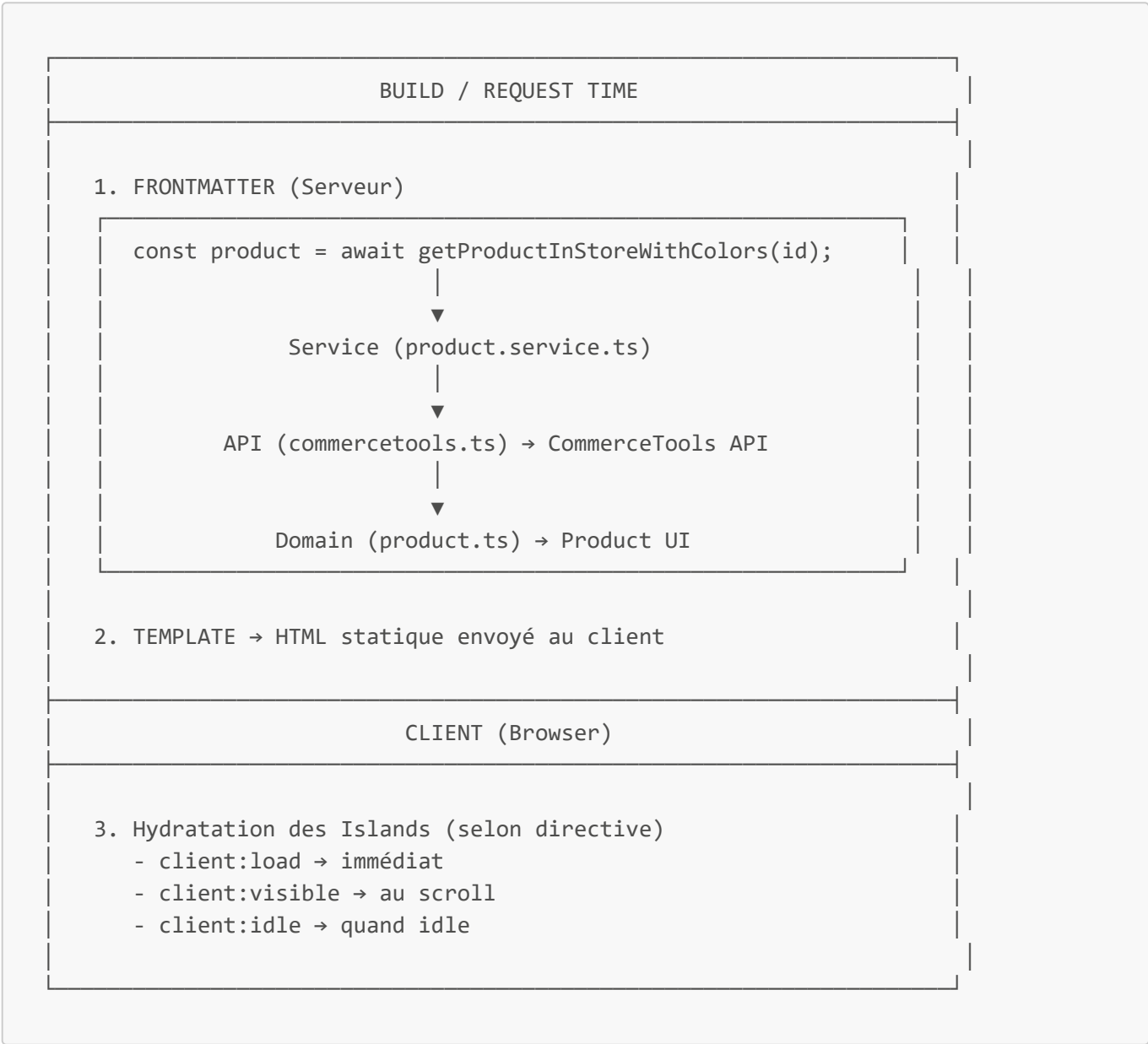
Anatomie d'une page Astro

```
---  
// === FRONTMATTER (exécuté côté serveur) ===  
import Layout from "../layouts/Layout.astro";  
import { getProductInStoreWithColors } from "../lib/services/product.service";  
import { ProductGallery } from "../features/product/components/ProductGallery";  
import { ProductInteractive } from  
  "../features/product/components/ProductInteractive";  
  
// Récupération des paramètres URL  
const { id } = Astro.params;  
  
// Fetch des données côté serveur (SSR)  
const product = await getProductInStoreWithColors(id);  
  
if (!product) {  
  return Astro.redirect("/404");  
}  
---  
  
<!-- === TEMPLATE (HTML généré) === -->  
<Layout title={product.name}>  
  <!-- Composant statique (pas de JS) -->  
  <h1>{product.name}</h1>  
  <p>{product.description}</p>  
  
  <!-- Island : hydraté quand visible (lazy) -->  
  <ProductGallery  
    images={product.images}  
    client:visible  
  />  
  
  <!-- Island : hydraté immédiatement (critique) -->  
  <ProductInteractive  
    product={product}  
    client:load  
  />  
</Layout>
```

Directives d'hydratation

Directive	Quand utiliser	Exemple
client:load	Critique, visible immédiatement	Bouton Add to Cart
client:visible	Peut attendre d'être visible	Galerie images, Carousel
client:idle	Basse priorité	Accordéons, FAQ
client:only="preact"	Jamais de SSR	Devtools, composants browser-only
(rien)	Statique, pas d'interactivité	Headers, textes, images

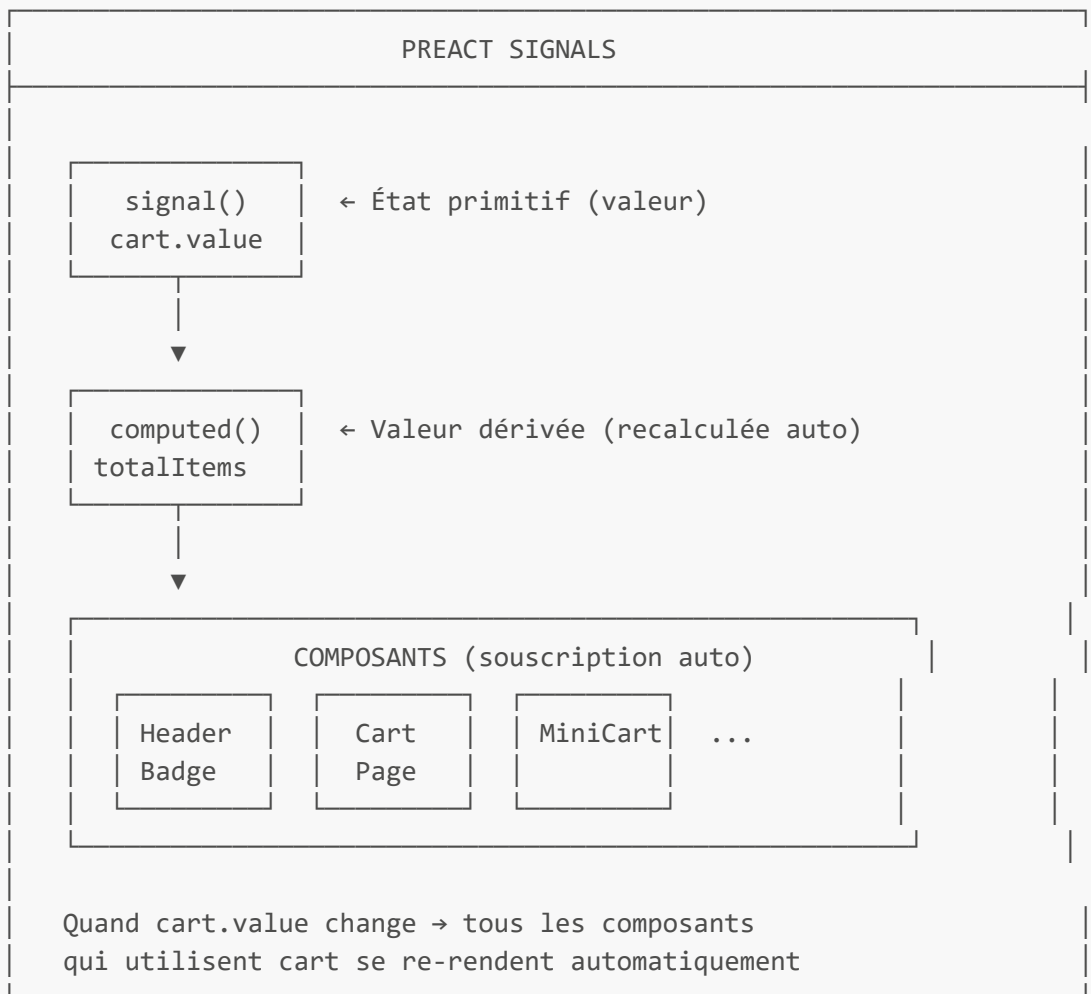
Flux de données : Page Astro



5. Preact Signals : état client

Concept

Les **Signals** sont des valeurs réactives. Quand un signal change, les composants qui l'utilisent se re-rendent automatiquement.



Structure d'un store

```
// src/lib/signals/cart.store.ts

import { signal, computed } from "@preact/signals";
import type { Cart, CartItem } from "../domain/cart";

// =====
// STATE (Signals) - Valeurs réactives
// =====

export const cart = signal<Cart | null>(null);
export const loading = signal<boolean>(false);
export const error = signal<string | null>(null);

// =====
// COMPUTED - Valeurs dérivées (recalculées automatiquement)
// =====
```

```

export const totalQuantity = computed(() : number => {
  if (!cart.value) {
    return 0;
  }
  return cart.value.items.reduce((sum, item) => sum + item.quantity, 0);
});

export const isEmpty = computed(() : boolean => {
  return totalQuantity.value === 0;
});

// =====
// ACTIONS - Fonctions qui modifient l'état
// =====

export async function addToCart(data: AddToCartData): Promise<boolean> {
  loading.value = true;
  error.value = null;

  try {
    const res = await fetch("/api/cart/add", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });

    const result = await res.json();

    if (!res.ok) {
      error.value = result.error;
      return false;
    }

    cart.value = result.cart; // ← Mise à jour du signal
    return true;
  } catch (err) {
    error.value = "network_error";
    return false;
  } finally {
    loading.value = false;
  }
}

```

Utilisation dans un composant

```

// src/features/cart/CartBadge.tsx

import { totalQuantity, loading } from "../../lib/signals/cart.store";

export function CartBadge() {
  // Accès direct au signal → souscription automatique

```

```
if (loading.value) {
  return <span class="badge">...</span>;
}

return (
  <span class="badge">
    {totalQuantity.value}  {/* Re-render auto si totalQuantity change */}
  </span>
);
}
```

Règle importante

⚠ SIGNALS = CLIENT ONLY

Les signals ne fonctionnent que côté client (browser).

☒ CORRECT

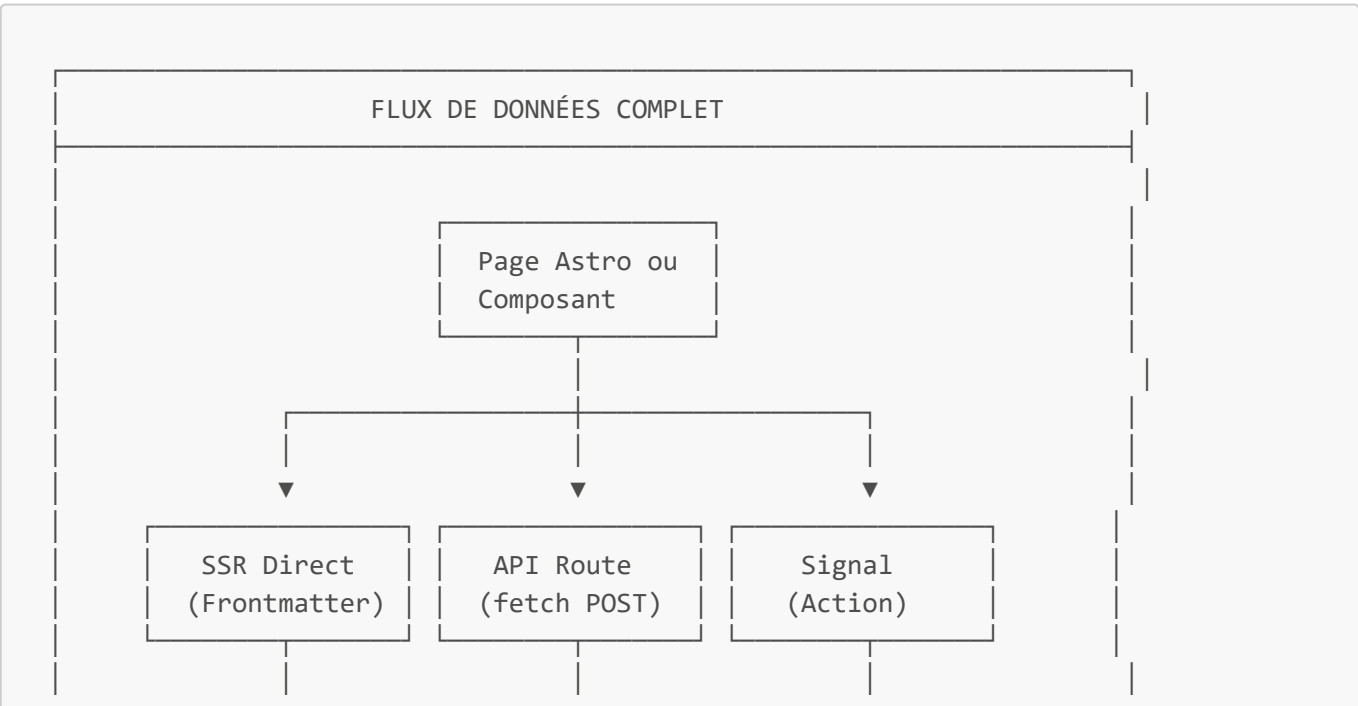
☐ INTERDIT

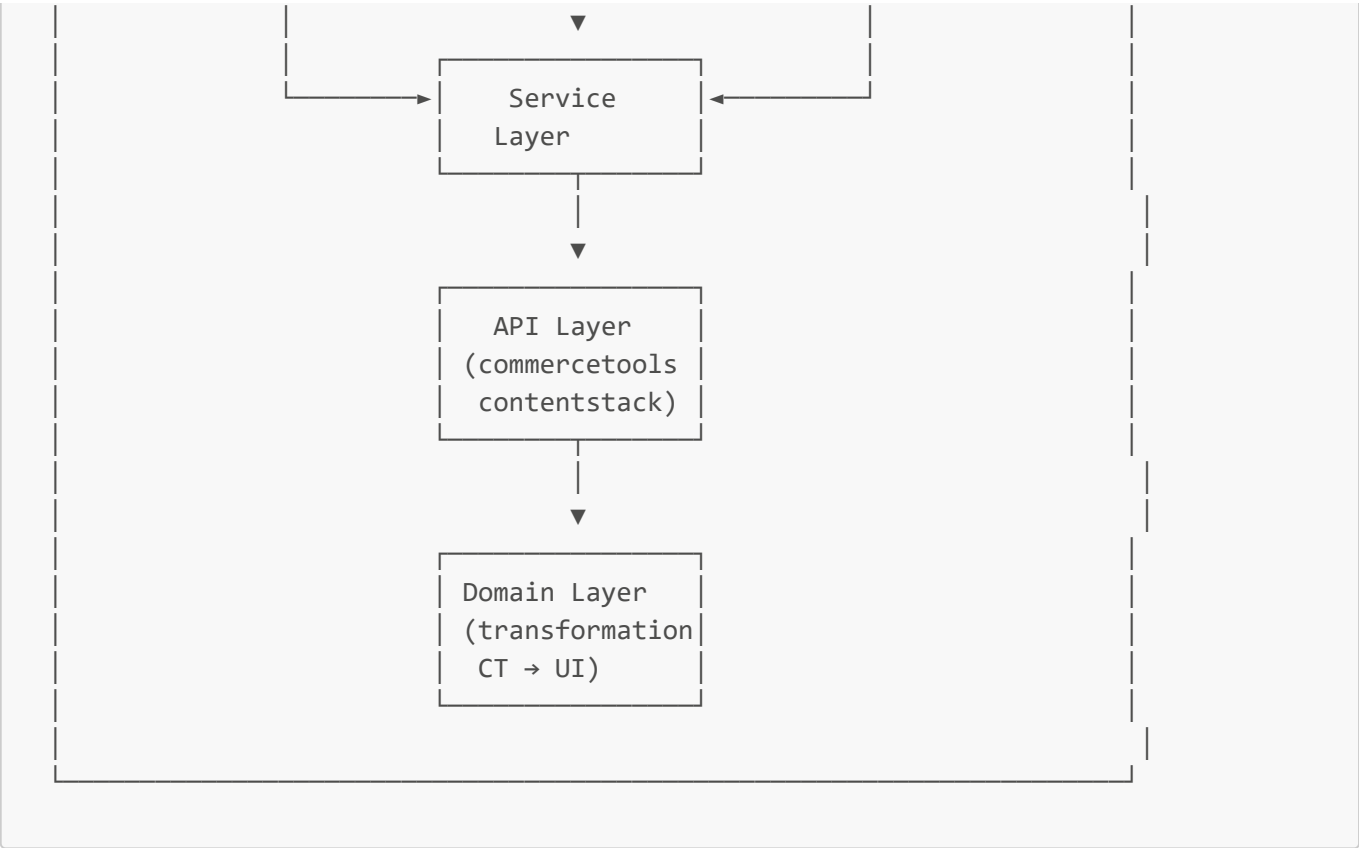
components/*.tsx
features/**/*.tsx
lib/signals/*.store.ts

pages/*.astro (frontmatter)
lib/services/*.ts
lib/api/*.ts

6. Récupération des données

Vue d'ensemble





Pattern 1 : SSR Direct (Page Astro)

Pour les données nécessaires au rendu initial (SEO, First Paint).

```
---
// pages/product/[id].astro
import { getProductInStoreWithColors } from "../../lib/services/product.service";

const { id } = Astro.params;

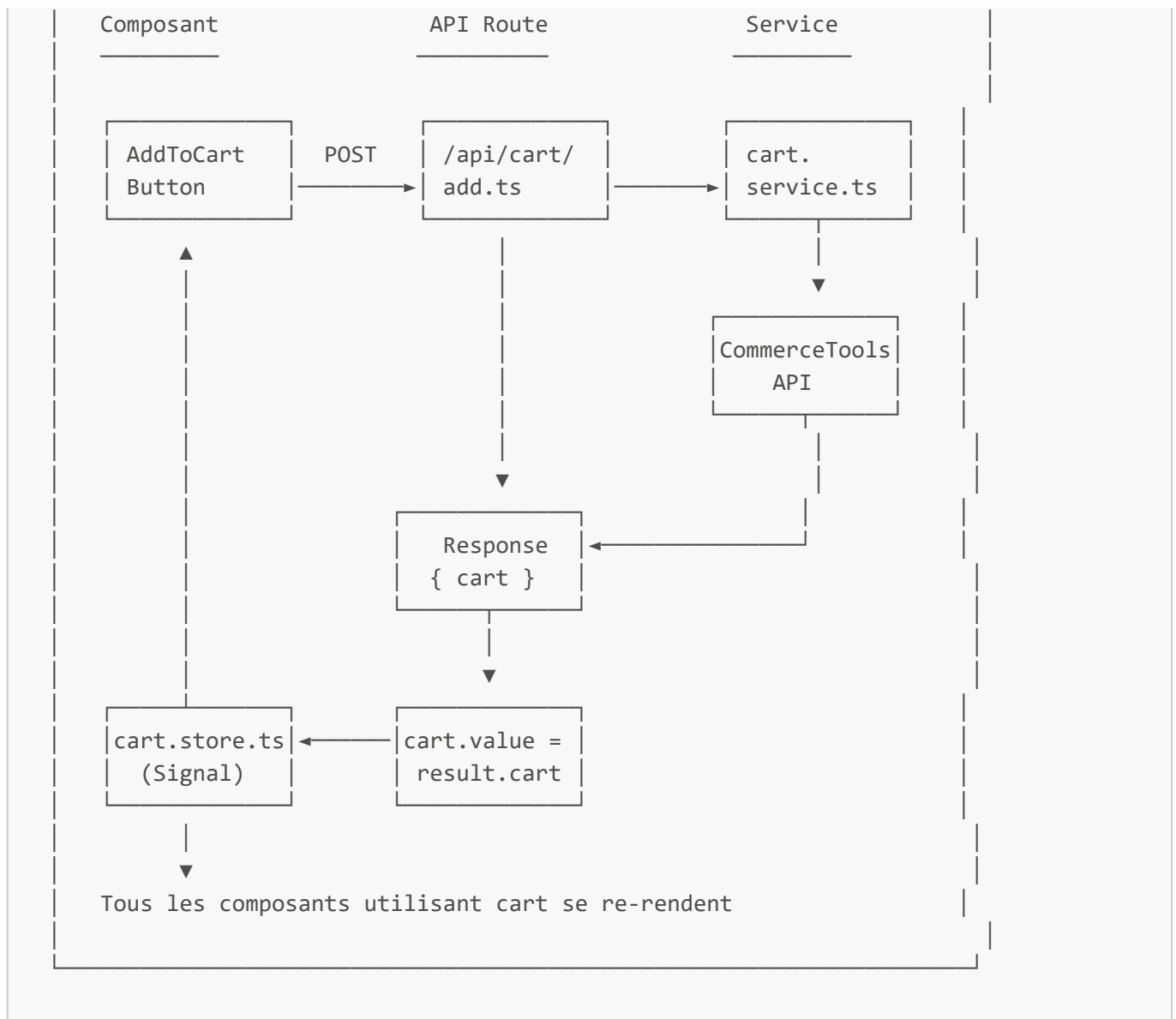
// Fetch côté serveur (jamais exposé au client)
const product = await getProductInStoreWithColors(id);
---

<Layout>
  <h1>{product.name}</h1>
  <p>Prix: {product.price} {product.currency}</p>
</Layout>
```

Pattern 2 : API Route + Signal (Actions client)

Pour les interactions utilisateur (add to cart, login, etc.).





Exemple concret :

```

// 1. Signal Store (cart.store.ts)
export async function addToCart(data: AddToCartData): Promise<boolean> {
  loading.value = true;

  const res = await fetch("/api/cart/add", {
    method: "POST",
    body: JSON.stringify(data),
  });

  const result = await res.json();
  cart.value = result.cart; // ← Update signal

  loading.value = false;
  return true;
}

// 2. API Route (pages/api/cart/add.ts)
export async function POST({ request }) {
  const body = await request.json();

```

```

    const { productId, variantId, quantity } = body;

    // Appel du service (côté serveur)
    const cart = await addToCartService(productId, variantId, quantity);

    return new Response(JSON.stringify({ cart }));
}

// 3. Service (lib/services/cart.service.ts)
export async function addToCartService(productId, variantId, quantity) {
    // Appel CommerceTools API
    const ctCart = await addLineItemInStore(storeKey, cartId, {
        productId,
        variantId,
        quantity,
    });

    // Transformation vers format UI
    return mapCTCart(ctCart);
}

```

Pattern 3 : useEffect (Fetch au mount)

Pour charger des données après l'hydratation.

```

import { useEffect, useState } from "preact/hooks";

export function CrossSellBlock({ categoryKey }) {
    const [products, setProducts] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        async function fetchProducts() {
            const res = await fetch(`/api/crosssell?category=${categoryKey}`);
            const data = await res.json();
            setProducts(data.products);
            setLoading(false);
        }

        fetchProducts();
    }, [categoryKey]);

    if (loading) {
        return <Loader />;
    }

    return (
        <div class="grid">
            {products.map(product => (
                <ProductCard key={product.id} {...product} />
            ))}
        </div>
    );
}

```

```
    </div>
  );
}
```

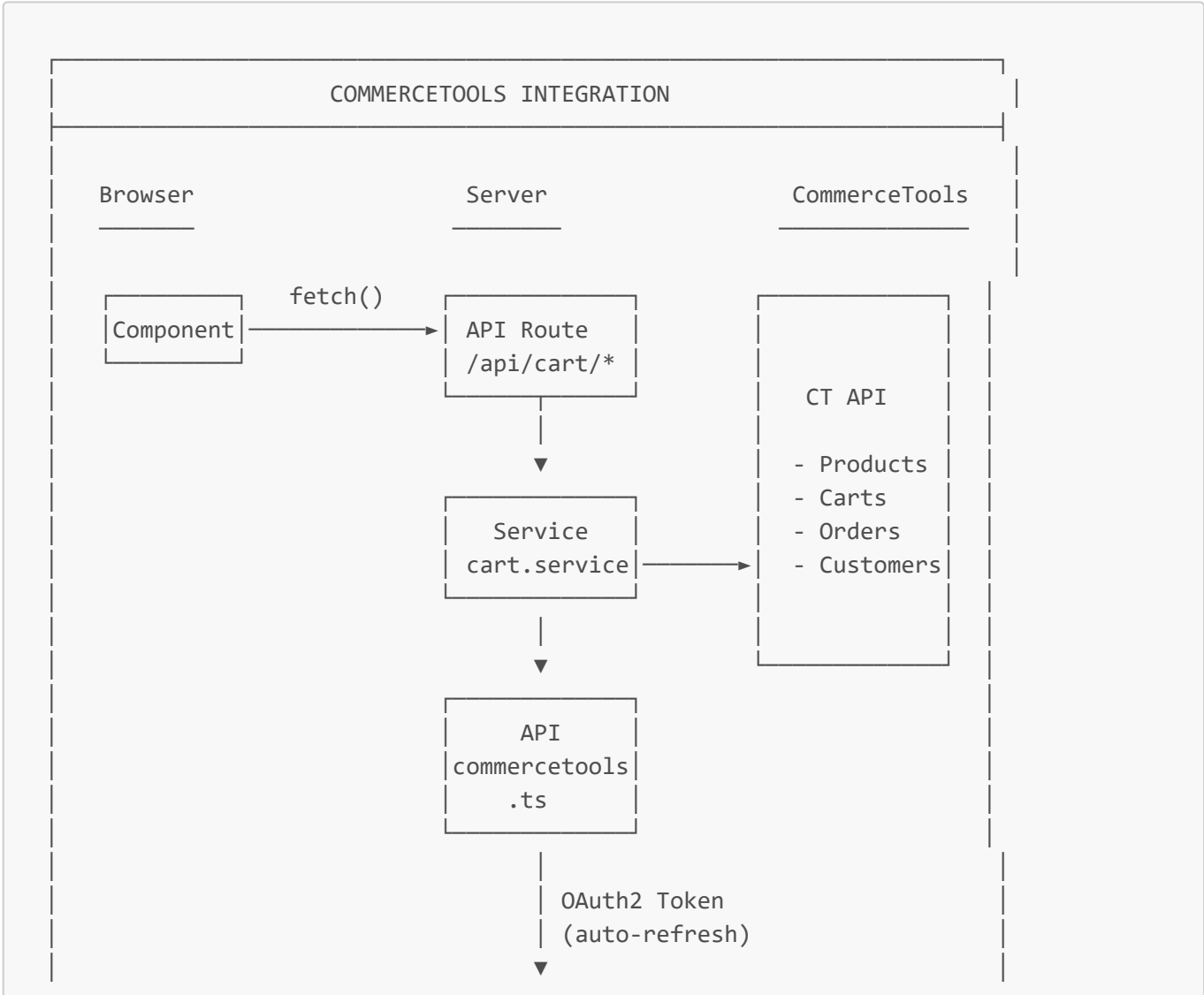
Quand utiliser quoi ?

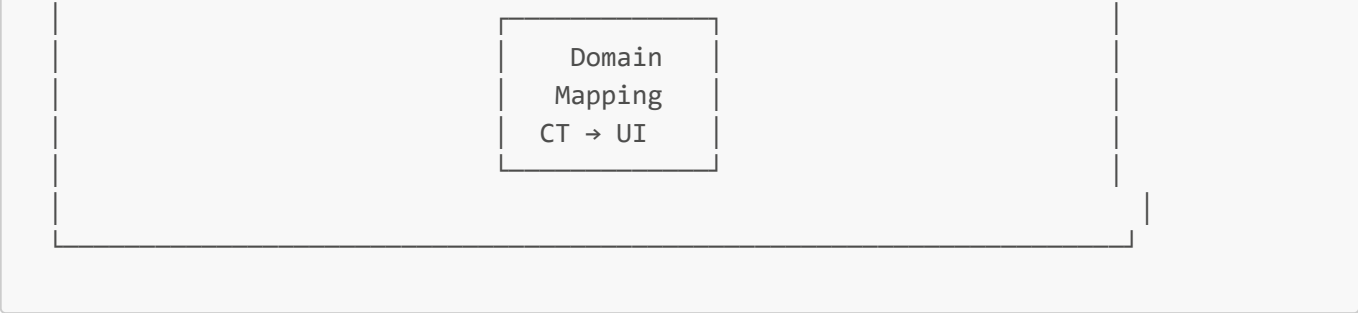
Besoin	Pattern	Exemple
Données pour SEO/First Paint	SSR Direct (Astro)	Fiche produit, listing
Action utilisateur	Signal + API Route	Add to cart, login
Données non-critiques	useEffect	Recommendations, cross-sell
État partagé multi-composants	Signal store	Cart, user session

7. Connexion CommerceTools

Note : À terme, les appels passeront par un BFF (Back-to-Front). Cette section décrit l'architecture actuelle.

Architecture actuelle





Domain
Mapping
CT → UI

Configuration multi-marques

```
// src/config/brands.ts

export const brandConfigs = {
  etam: {
    ct: {
      storeKey: "ETAM_FR",
      distributionChannelId: "b9d48b47-...",
      productSelectionId: "3fa85450-...",
      currency: "EUR",
      country: "FR",
    }
  },
  undiz: {
    ct: {
      storeKey: "UNDIZ_FR",
      // ...
    }
  }
};
```

Couches d'abstraction

1. **API Layer** (`lib/api/commercetools.ts`) - Appels HTTP bruts
2. **Service Layer** (`lib/services/*.service.ts`) - Orchestration
3. **Domain Layer** (`lib/domain/*.ts`) - Transformation CT → UI

```
// Exemple : Récupération d'un produit

// 1. Service (appelé par Astro ou API Route)
const product = await getProductInStoreWithColors(productId);

// 2. Le service utilise l'API layer
async function getProductInStoreWithColors(id) {
  const projection = await fetchProductProjectionById(id); // API
  return mapCTProductProjection(projection); // Domain
}

// 3. L'API layer appelle CommerceTools
async function fetchProductProjectionById(id) {
```

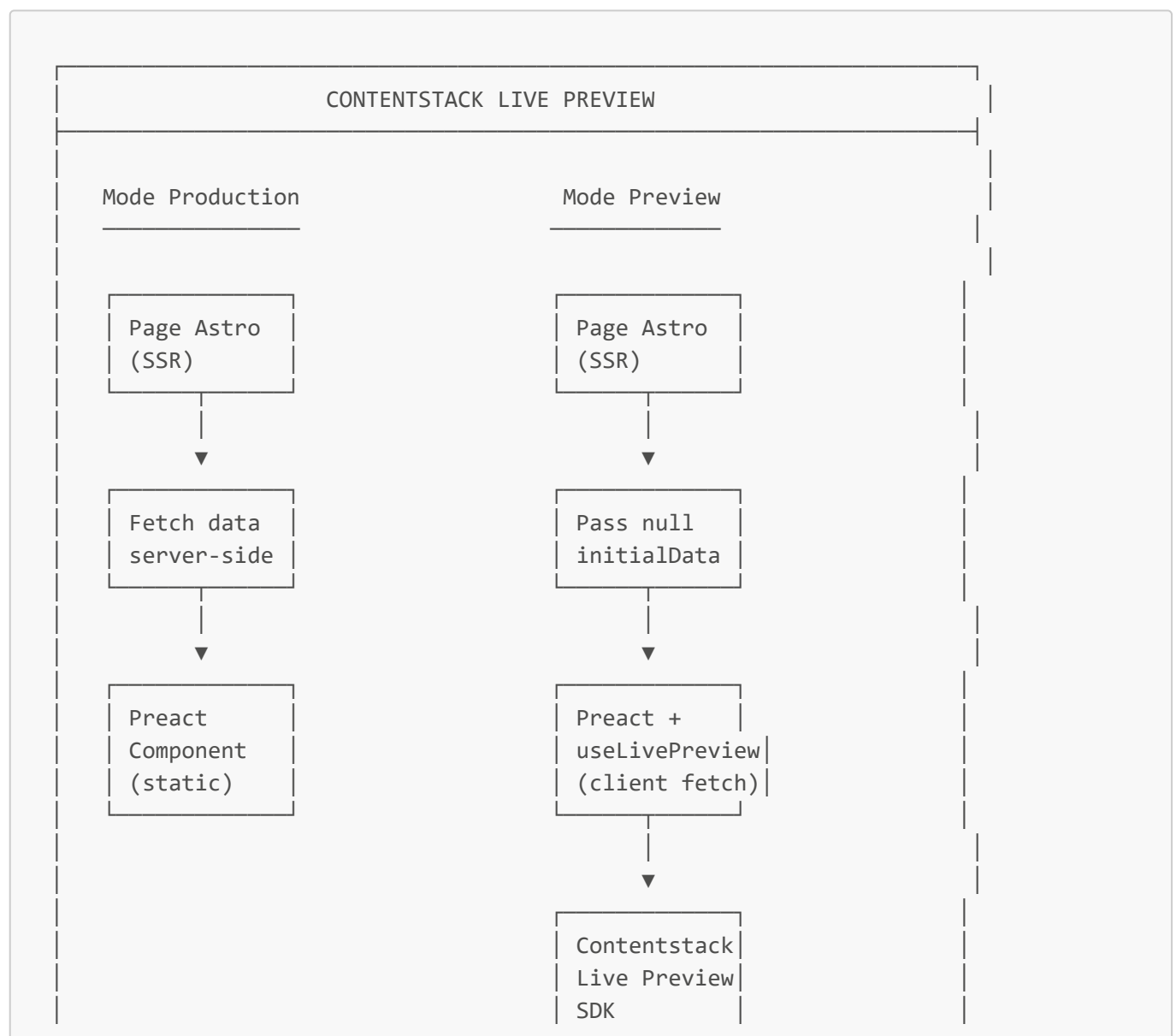
```
    return ctFetch(`/product-projections/${id}`);
  }

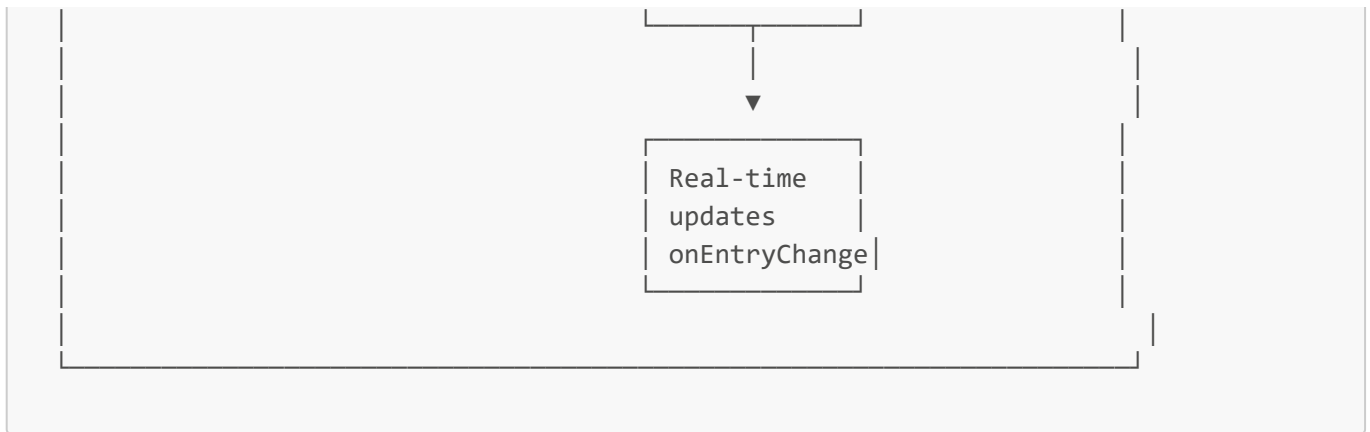
  // 4. Le domain transforme les données
  function mapCTProductProjection(ct) {
    return {
      id: ct.id,
      name: ct.name["fr-FR"],
      price: ct.masterVariant.prices[0].value.centAmount / 100,
      // ...
    };
  }
}
```

8. Live Preview Contentstack

Concept

Contentstack permet aux contributeurs de voir leurs modifications **en temps réel** dans le site, sans déploiement.





Implémentation

1. Page Astro (index.astro)

```

---
import { getHomePageEntry } from "../lib/services/content.service";
import HomePageContent from "../features/homepage/HomePageContent";

// Détection du mode preview
const isPreview = import.meta.env.PUBLIC_IS_PREVIEW === "true";

// En production : fetch server-side
// En preview : laisser le client fetcher
const homeEntry = !isPreview ? await getHomePageEntry() : null;
---

<Layout>
  <HomePageContent
    brand="etam"
    isPreview={isPreview}
    initialData={homeEntry}
    client:load
  />
</Layout>

```

2. Composant avec Live Preview (HomePageContent.tsx)

```

import { useLivePreview } from "../../lib/contentstack-sdk/useLivePreview";

export default function HomePageContent({ brand, isPreview, initialData }) {
  // Hook qui gère SSR et Live Preview
  const { entry, isLoading, error } = useLivePreview<HomePage>({
    contentType: "homepage",
    fieldName: "brand",
    fieldValue: brand,
    isPreview, // Active le mode preview
    initialData, // Données SSR (null en preview)
  });
}

```

```

    if (isLoading) return <Loader />;
    if (error) return <Error message={error} />;

    return (
      <div>
        {entry.blocks.map(block => (
          <Block key={block.id} {...block} />
        ))}
      </div>
    );
  }
}

```

3. Hook useLivePreview

```

export function useLivePreview<T>({ contentType, fieldName, fieldValue, isPreview,
initialData }) {
  const [entry, setEntry] = useState<T | null>(initialData ?? null);
  const [isLoading, setIsLoading] = useState(isPreview && !initialData);

  useEffect(() => {
    if (!isPreview) return; // Mode production : rien à faire

    async function initLivePreview() {
      // Initialisation du SDK Contentstack Live Preview
      const { default: ContentstackLivePreview } = await
import("@contentstack/live-preview-utils");

      await ContentstackLivePreview.init({
        mode: "builder",
        editButton: { enable: true },
        // ...
      });

      // Écoute des changements en temps réel
      ContentstackLivePreview.onEntryChange(() => {
        fetchData(); // Re-fetch à chaque modification
      });

      fetchData();
    }

    initLivePreview();
  }, [isPreview]);

  return { entry, isLoading, error };
}

```

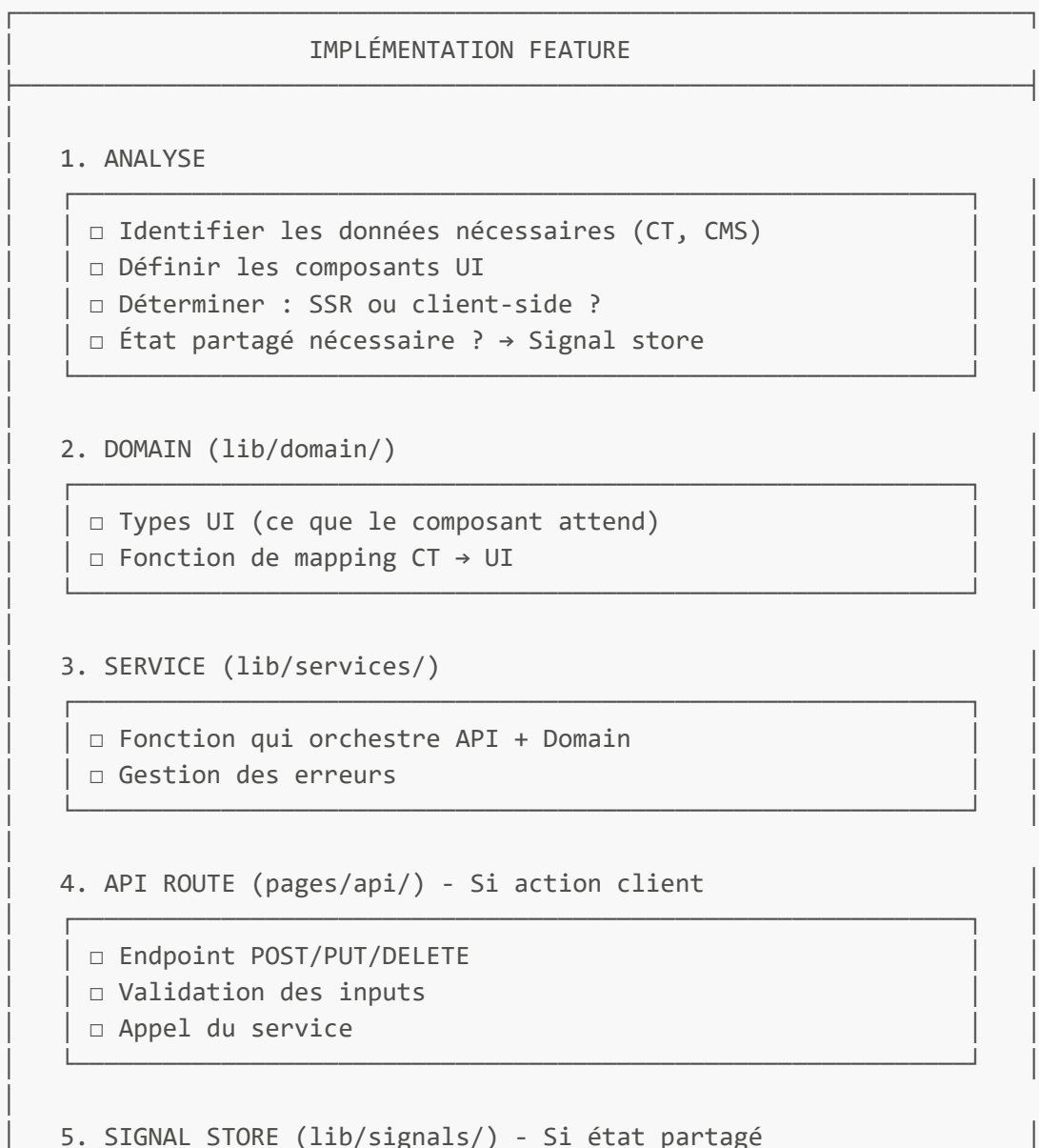
Fonctionnalités

- **Visual Builder** : Click-to-edit sur les éléments
- **Real-time sync** : Modifications visibles instantanément
- **Editable props** : Attributs spéciaux pour identifier les champs éditables

```
// Ajout des props éditables pour Visual Builder
<FullBanner
  title={banner.title}
  editable={{
    titleProps: getEditableProps(banner, "title"),
    subtitleProps: getEditableProps(banner, "subtitle"),
  }}
/>
```

9. Méthodologie d'implémentation

Checklist nouvelle feature



- Signals (état)
- Computed (dérivés)
- Actions (fetch via API routes)

6. COMPOSANTS (features/ ou components/)

- Props.ts (types)
- Component.tsx
- CSS si nécessaire

7. PAGE ASTRO (pages/)

- Fetch SSR dans frontmatter
- Directive d'hydratation appropriée
- Layout + SEO

Exemple complet : Wishlist

```
// 1. DOMAIN - lib/domain/wishlist.ts
export interface WishlistItem {
  productId: string;
  variantId: string;
  name: string;
  price: number;
}

export function mapCTWishlist(ctCart): WishlistItem[] {
  return ctCart.lineItems.map(item => ({
    productId: item.productId,
    variantId: item.variant.id,
    name: item.name["fr-FR"],
    price: item.price.value.centAmount / 100,
  }));
}

// 2. SERVICE - lib/services/wishlist.service.ts
export async function addToWishlist(productId, variantId) {
  const cart = await addLineItemToWishlistCart(productId, variantId);
  return mapCTWishlist(cart);
}

// 3. API ROUTE - pages/api/wishlist/add.ts
export async function POST({ request }) {
  const { productId, variantId } = await request.json();
  const items = await addToWishlist(productId, variantId);
}
```

```
    return new Response(JSON.stringify({ items }));
  }

// 4. SIGNAL STORE - lib/signals/wishlist.store.ts
export const wishlistItems = signal<WishlistItem[]>([]);

export async function addToWishlistAction(productId, variantId) {
  const res = await fetch("/api/wishlist/add", {
    method: "POST",
    body: JSON.stringify({ productId, variantId }),
  });
  const { items } = await res.json();
  wishlistItems.value = items;
}

// 5. COMPOSANT - features/wishlist/WishlistButton.tsx
import { addToWishlistAction } from "../../lib/signals/wishlist.store";

export function WishlistButton({ productId, variantId }) {
  return (
    <button onClick={() => addToWishlistAction(productId, variantId)}>
      <Icon name="heart" />
    </button>
  );
}

// 6. PAGE ASTRO - Intégration
<WishlistButton
  productId={product.id}
  variantId={product.variantId}
  client:load
/>
```

Commandes utiles

```
# Développement par marque
npm run dev:etam      # Port 4321
npm run dev:undiz     # Port 4322
npm run dev:maison123 # Port 4323

# Build
npm run build:etam
npm run build:undiz

# Linting
npm run lint          # ESLint + Stylelint
```

Ressources

- [Documentation Astro](#)
 - [Preact Signals](#)
 - [CommerceTools API](#)
 - [Contentstack Live Preview](#)
-

Questions ? N'hésitez pas à demander à l'équipe. Bienvenue sur le projet !